

# ScheduRL

401170553-401170491

**MohammadAmin Heidari**

**Mani Ebrahimi**

## 1. Project Overview & Motivation

The proliferation of Internet of Things (IoT) devices has led to a surge in time-sensitive data processing requirements. However, IoT devices are inherently resource-constrained, often lacking the computational power to execute complex tasks within tight deadlines. **Edge Computing** offers a middle ground between local execution and high-latency Cloud processing by placing compute resources at the network's periphery.

The primary challenge addressed by the **ScheduRL** framework is the management of **Hybrid Workloads**. These workloads consist of:

1. **Periodic Tasks:** Hard real-time tasks (e.g., sensor polling, heartbeat signals) that occur at fixed intervals and must meet their deadlines to ensure system stability.
2. **Soft Tasks (Aperiodic):** Dynamic tasks (e.g., image recognition, data burst processing) that arrive unpredictably. While they have deadlines, the system aims for "Best Effort" optimization to maximize the success rate.

## The IoT-Edge Trade-off

The framework models the fundamental trade-off between **Local Execution** and **Edge Offloading**:

- **Local Execution:** Low latency (no network transfer) but slow execution due to limited CPU.
- **Edge Offloading:** High compute power (fast execution) but introduces network transfer overhead and potential congestion at the server.

## 2. System Pipeline

The ScheduRL framework follows a structured execution pipeline managed by `main.py` and implemented within `services/iot_edge_experiment.py`.

### 2.1 Initialization and Setup

The process begins when `main.py` invokes `run_iot_edge_experiment`. The environment is initialized by building a spatial topology where devices and servers are placed on a 2D

coordinate system. Connectivity is determined based on a `range_limit` (35.0 units), simulating the physical constraints of wireless communication.

## 2.2 Task Generation Lifecycle

Tasks are generated using specialized utility functions:

- `_generate_periodic_tasks` : Creates a baseline of hard real-time requirements for each device.
- `_generate_soft_tasks` : Simulates dynamic system load.

```
soft_tasks = []
for device in devices:

    num_tasks = rng.randint(int(5 * load_scale), int(15 * load_scale))
    for _ in range(num_tasks):
        soft_tasks.append(SoftTask(
            task_id=f"soft_{device.device_id}_{len(soft_tasks)}",
            device_id=device.device_id,
            arrival_time=rng.uniform(0, 100),
            execution_time=rng.uniform(0.5, 2.5),
            relative_deadline=rng.uniform(3.0, 8.0),
            data_size_mb=rng.uniform(1.0, 5.0),
            result_size_mb=rng.uniform(0.1, 0.5)
        ))
```

## 2.3 The Simulation Loop ( `_simulate` )

The `_simulate` function acts as the discrete-event engine. It processes tasks in chronological order. For every soft task arrival, the selected **Optimization Algorithm** (None, GA, or RL) decides whether the task stays on the device or moves to an available Edge Server.

## 2.4 Visualization and Reporting

Post-simulation, `iot_view.py` processes the `DecisionRecord` objects to generate Gantt charts. These charts provide a visual validation of task overlaps, offloading decisions, and deadline compliance.

# 3: CLI Variables, Environment Setup, and Algorithm Logic

## 3.1 Terminal Variables & Environment Control

The behavior of the simulation is controlled by two primary flags: `--mode` and `--vars` . These determine the physical layout of your IoT network.

## The `--vars` Variable System

The `--vars` argument is a dynamic parameter. Its meaning shifts depending on which experimental mode you select, allowing you to stress-test different bottlenecks:

| Mode ( <code>--mode</code> ) | Meaning of <code>--vars N</code> | Practical Scenario                                                                                                                              |
|------------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>staticserver</code>    | <b>N IoT Devices</b>             | <b>Device Congestion:</b> A smart factory with a fixed number of edge gateways but a growing number of sensors.                                 |
| <code>iot</code>             | <b>N Edge Servers</b>            | <b>Infrastructure Scaling:</b> A city-wide deployment where 100 devices are fixed, and you want to see if adding more servers improves latency. |

## The `--alg` Selection

This flag defines the logic used by the system to handle soft-task offloading.

- **Syntax:** `python main.py --mode [mode] --vars [N] --alg [none|gen|rl]`
- 

### 3.2 Deep Dive: The Three Algorithms ( `--alg` )

The framework implements three distinct strategies for making real-time offloading decisions. Every algorithm is constrained by the **Hard Schedulability Rate**—meaning they cannot offload or execute tasks in a way that breaks the local **EDF (Earliest Deadline First)** schedule for periodic tasks.

#### 1. Algorithm: `none` (The Control Baseline)

In this mode, the system operates without edge assistance for dynamic workloads.

- **Logic:** Every soft task is kept on the device where it arrived.
- **Decision Process:** There is no decision logic; the server ID is always `None`.
- **Performance:** It serves as a "worst-case" baseline for compute-intensive tasks. You will notice high **Latency** and a low **Soft Success Rate** because the IoT device's CPU becomes the bottleneck.
- **Use Case:** Ideal for demonstrating why Edge Computing is necessary in high-load scenarios.

#### 2. Algorithm: `gen` (Genetic Algorithm Optimization)

The `gen` mode uses an evolutionary strategy to find a near-optimal distribution of tasks across all available servers.

- **The Chromosome:** Decisions are encoded as a vector of integers (e.g.,  $[0, 2, 1, 0]$ ), where each position is a task and each value is the assigned server (0 = local).
- **Fitness Function:** The algorithm evaluates "fitness" based on the formula:

$$Fitness = W_1 \cdot (\text{Success Rate}) - W_2 \cdot (\text{Avg Latency}) - W_3 \cdot (\text{Energy})$$

- **Execution:** Before the simulation starts its online phase, the GA runs through generations of selection, crossover, and mutation to find the best offloading map.
- **Pros/Cons:** It finds excellent schedules for static batches but is computationally expensive, leading to a higher "Decision Budget" overhead.

### 3. Algorithm: rl (Reinforcement Learning - Q-Learning)

The rl mode uses a model-free Q-Learning agent that learns through environmental interaction.

- **The State ( $S$ ):** A tuple representing (Source\_Device\_ID, Task\_Workload\_Size, Current\_Server\_Load) .
- **The Action ( $A$ ):** A discrete choice from  $\{0, 1, \dots, N\}$  representing which node should process the task.
- **The Reward ( $R$ ):**
  - **Success:** +10 reward if the task completes before its absolute deadline.
  - **Failure:** -5 penalty if the deadline is missed.
  - **Overhead:** A small negative penalty proportional to the transfer\_overhead to discourage unnecessary offloading.
- **The Q-Table:** A lookup table that the agent updates using the Bellman Equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- **Pros/Cons:** It is highly adaptive. While it may struggle in the first 10% of the simulation (exploration), it eventually learns the network's "congestion patterns" and outperforms GA in dynamic, unpredictable workloads.

### 3.3 Comparative Summary

| Feature      | none             | gen                      | rl                         |
|--------------|------------------|--------------------------|----------------------------|
| Complexity   | Low              | High                     | Medium                     |
| Adaptability | None             | Low (Re-run required)    | High (Continuous learning) |
| CPU Overhead | Minimal          | Very High (Optimization) | Low (Table lookup)         |
| Best For     | Baseline testing | Small, fixed task sets   | Large, dynamic deployments |

---

## 3.4 How to Run and Compare

To see the difference between these algorithms, you can run a comparison by omitting the `-alg` flag (which triggers the comparison loop in the code):

Bash

```
# Run a comparison across all 3 algs for a fleet of 5 servers and 100 devices
python main.py --mode iot --vars 5
```

This will generate a series of charts in the `plots/` directory, comparing the **Deadline Success Rate** and **Makespan** of `none`, `gen`, and `rl` side-by-side.

Here is the comprehensive technical continuation of the ScheduRL framework report, deeply analyzing the core scheduling mechanisms, the optimization algorithms, and an exhaustive file-by-file breakdown.

---

## 3. Core Scheduling Algorithms

The ScheduRL framework relies on established real-time scheduling theory to guarantee resource availability before applying artificial intelligence for offloading decisions. The two foundational algorithms used are **Earliest Deadline First (EDF)** and the **Improved Total Bandwidth Server (iTBS)**.

### 3.1 EDF (Earliest Deadline First)

EDF is an optimal, dynamic-priority scheduling algorithm used during the framework's **Offline Phase**. Its primary role is to ensure that the strict, hard real-time periodic tasks generated by the IoT devices are schedulable on the local CPU without external edge server assistance.

If a device cannot schedule its own hard tasks, it is deemed unschedulable. The framework simulates EDF preemptively using a priority queue ( `heapq` ) to always execute the ready task with the closest absolute deadline.

**Implementation Snippet ( `iot_edge_experiment.py` ):**

The `_edf_schedulability_for_device` function simulates the EDF timeline over a specified `horizon`. It uses a min-heap to sort tasks by deadline.

Python

```

def _edf_schedulability_for_device(tasks: list[PeriodicTask], horizon:
float) -> tuple[bool, list[Segment]]:
    # 1. Pre-calculate all task releases within the simulation horizon
    releases = []
    for idx, task in enumerate(tasks):
        t = task.release_offset
        while t <= horizon + 1e-9:
            releases.append((t, idx))
            t += task.period
    releases.sort(key=lambda x: x[0])

    ready = [] # The Priority Queue (Min-Heap)
    segments = []
    miss = False
    now = 0.0
    release_cursor = 0

    while release_cursor < len(releases) or ready:
        # 2. Push all tasks that have arrived by 'now' into the ready heap
        while release_cursor < len(releases) and releases[release_cursor][0]
<= now + 1e-9:
            release_t, idx = releases[release_cursor]
            task = tasks[idx]
            deadline = release_t + task.period
            # Heap is sorted by the first tuple element: deadline
            heapq.heappush(ready, (deadline, idx, task.execution_time,
task.task_id))
            release_cursor += 1

            if not ready:
                now = releases[release_cursor][0]
                continue

        # 3. Pop the task with the Earliest Deadline
        deadline, idx, remaining, task_id = heapq.heappop(ready)
        if now > deadline + 1e-9:
            miss = True # Deadline missed!
            break

        # Execute until completion or until the next task release
        next_release = releases[release_cursor][0] if release_cursor <
len(releases) else float("inf")
        run = min(remaining, next_release - now)

        # ... (record segment and update time)

```

## 3.2 iTBS (Improved Total Bandwidth Server)

While EDF handles hard periodic tasks, **iTBS** manages the dynamic, unpredictable influx of **Soft Aperiodic Tasks**. iTBS is utilized heavily by the Edge Servers ( `schedule_itbs` in `scheduler_service.py` ) to assign virtual deadlines to incoming offloaded tasks based on a dedicated server bandwidth portion ( $U_s$ ).

The iTBS algorithm isolates aperiodic tasks from periodic tasks by assigning a virtual deadline  $d_k$  to the  $k$ -th aperiodic job arriving at time  $r_k$  with execution time  $C_k$ :

$$d_k = \max(r_k, d_{k-1}) + \frac{C_k}{U_s}$$

### Idle Time Reclamation:

The "Improved" aspect of iTBS comes from its ability to reclaim idle time. If the server has finished its previous tasks and sits idle, the virtual deadline accumulator ( $d_{k-1}$ ) is reset to the current arrival time ( $r_k$ ). Without this, a server that was heavily loaded in the past would artificially penalize new tasks arriving hours later.

### Implementation Snippet ( `scheduler_service.py` ):

```
def schedule_itbs(jobs, server_bandwidths, initial_deadline=0.0,
reclaim_idle_time=True):
    # ... validation logic ...

    for job in sorted_jobs:
        # Select the server that can provide the earliest finish/deadline
        selected_server_id = min(...)
        selected_bandwidth = normalized_bandwidths[selected_server_id]
        previous_virtual_deadline =
previous_virtual_deadlines[selected_server_id]
        previous_finish_time = previous_finish_times[selected_server_id]

        # IDLE TIME RECLAMATION LOGIC
        if reclaim_idle_time and job.arrival_time > previous_finish_time:
            # Server was idle; reset the virtual deadline to the job's
arrival time
            previous_virtual_deadline = max(initial_deadline,
job.arrival_time)

        # Calculate the new assigned virtual deadline
        virtual_start = max(job.arrival_time, previous_virtual_deadline)
        assigned_deadline = virtual_start + (job.execution_time /
selected_bandwidth)

        service_start = max(job.arrival_time, previous_finish_time)
        service_finish = service_start + job.execution_time

        # Update trackers
        previous_virtual_deadlines[selected_server_id] = assigned_deadline
```

```
previous_finish_times[selected_server_id] = service_finish
# ... append results ...
```

## 4. Optimization Algorithms

During the **Online Phase**, when a soft task arrives, the system must decide whether to execute it locally or offload it. This decision is driven by either a Genetic Algorithm (GA) or Reinforcement Learning (RL).

### 4.1 Genetic Algorithm (GA)

Unlike traditional GA implementations that directly map tasks to servers, ScheduRL implements an elegant **Heuristic Weight-Tuning GA** ( `_train_ga_weights` ).

Because online scheduling requires ultra-fast sub-millisecond decisions, running a full GA population evolution at *every task arrival* is computationally impossible. Instead, the framework extracts a subset of tasks to build "Feature Rows" and uses GA to evolve the optimal weights for a cost-evaluation heuristic.

- **Chromosome Representation:** A real-valued vector of three weights: `[w_t, w_e, w_m]`.
  - `w_t`: Importance of completion time.
  - `w_e`: Importance of energy consumption.
  - `w_m`: Penalty multiplier for missing the deadline.
- **Fitness Function:** The algorithm evaluates how well a specific set of weights minimizes the total systemic cost (time + energy + miss penalty) across a sample of 800 tasks.
- **Execution:**

```
def _train_ga_weights(rows: list[dict[str, float]], rng: random.Random)
-> tuple[float, float, float]:
    # Initialize population with random weights
    population = [[rng.uniform(0.7, 1.8), rng.uniform(0.05, 0.5),
rng.uniform(1.2, 3.5)] for _ in range(20)]

    def fitness(chrom: list[float]) -> float:
        w_t, w_e, w_m = chrom
        score = 0.0
        for row in rows:
            d = row["deadline"]
            local_miss = max(0.0, row["local_finish"] - d)
            off_miss = max(0.0, row["off_finish"] - d)

            # Calculate cost based on the chromosome's weights
            local_cost = (w_t * row["local_finish"]) + (w_e *
row["local_energy"]) + (w_m * local_miss)
```

```

        off_cost = (w_t * row["off_finish"]) + (w_e *
row["off_energy"]) + (w_m * off_miss)

        # The agent will choose the minimum cost; we want to
        minimize total systemic cost
        score += min(local_cost, off_cost)
        return -score # Negative because we sort descending (higher
        fitness is better)

    # ... standard GA selection, crossover, and mutation over 25
    generations ...

```

## 4.2 Reinforcement Learning (RL): Q-Learning

The RL agent uses a tabular Q-Learning approach ( `_train_q_table` ). It treats the offloading decision as a Markov Decision Process (MDP).

- **State Space ( $S$ ):** To prevent state-space explosion, continuous variables are discretized into bins in `_rl_state` . The state is a tuple ( `g_bin` , `s_bin` ) :
  - `g_bin` (Gain): The time saved by offloading ( `off_finish - local_finish` ). Discretized into 4 bins (e.g., highly negative = massive gain).
  - `s_bin` (Slack): The urgency of the task ( `deadline - min(local_finish, off_finish)` ). Discretized into 4 bins.
  - Total states =  $4 \times 4 = 16$ . This guarantees ultra-fast convergence.
- **Action Space ( $A$ ):** Binary choice.
  - 0 : Keep the task local.
  - 1 : Offload to the best available edge server.
- **Reward Function ( $R$ ):** Designed to heavily penalize missing deadlines while slightly penalizing energy and raw finish times to encourage efficiency.

$$R = -(\text{Finish Time} + 3.0 \times \text{Miss Penalty} + 0.25 \times \text{Energy})$$

**Implementation Snippet ( `iot_edge_experiment.py` ):**

```

def _train_q_table(rows: list[dict[str, float]], rng: random.Random) ->
dict[tuple[int, int], list[float]]:
    q: dict[tuple[int, int], list[float]] = {}
    alpha = 0.22 # Learning Rate
    gamma = 0.90 # Discount Factor
    epsilon = 0.28 # Exploration Rate

    for _ in range(20): # Training epochs
        for row in rows:
            # 1. Observe State
            state = _rl_state(row["local_finish"], row["off_finish"],
row["deadline"])

```

```

q_vals = q.setdefault(state, [0.0, 0.0])

# 2. Select Action (Epsilon-Greedy)
action = rng.randint(0, 1) if rng.random() < epsilon else (0 if
q_vals[0] >= q_vals[1] else 1)

# 3. Simulate Environment & Calculate Reward
if action == 0:
    finish, energy = row["local_finish"], row["local_energy"]
else:
    finish, energy = row["off_finish"], row["off_energy"]

miss = 1.0 if finish > row["deadline"] else 0.0
reward = -(finish + (3.0 * miss) + (0.25 * energy))

# 4. Update Q-Table (Bellman Equation)
q_vals[action] = q_vals[action] + alpha * (reward + gamma *
max(q_vals) - q_vals[action])

epsilon = max(0.05, epsilon * 0.9) # Decay exploration
return q

```

## 5. File-by-File Breakdown

The framework is highly modular, following a strict Model-View-Controller (MVC) and Service-Oriented Architecture.

### 5.1 Root and Entry Point

| File Name | Responsibility                                                                                                                                                                                                                                                                                                                |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| main.py   | The central entry point for the application. It parses CLI arguments ( argparse ), registers subcommands ( seed , list , clear , schedule , offload ), and routes the execution flow. It differentiates between running the experimental sandbox ( run_iot_edge_experiment ) and the database-backed task scheduler commands. |

### 5.2 Controllers ( controllers/ )

Controllers interpret user CLI input and coordinate with the Service layer.

| File Name | Responsibility |

| :--- | :--- |

| `command_controller.py` | A master router class ( `CommandController` ). It takes the parsed CLI arguments from `main.py` and delegates the execution to the specific sub-controllers below. |

| `schedule_controller.py` | Handles the `schedule` and `offload` CLI commands. It extracts tasks from the database or JSON, passes them to `scheduler_service.py` to get results, and passes those results to `schedule_view.py` for rendering. |

| `seed_controller.py` | Handles the `seed` command. Populates the SQLite database with a predefined list of realistic robotic/IoT tasks (from `seed_data.py` ). |

| `task_controller.py` | Handles basic CRUD operations for the CLI, specifically the `list` (show all tasks) and `clear` (delete all tasks) commands. |

## 5.3 Services ( `services/` )

This directory contains the core business logic, mathematics, and scheduling algorithms.

| File Name | Responsibility |

| :--- | :--- |

| `iot_edge_experiment.py` | **The most complex file in the project.** This executes the `--mode iot/staticserver` experiments. It handles dynamic topology generation ( `_build_topology` ), task generation, the EDF offline phase ( `_offline_phase` ), the GA optimization ( `_train_ga_weights` ), the RL optimization ( `_train_q_table` ), the online simulation loop ( `_simulate` ), and orchestrates the generation of Matplotlib comparison charts ( `_save_charts` ). |

| `scheduler_service.py` | The deterministic scheduling engine. Contains the pure mathematical implementations of `schedule_edf` (standard local Earliest Deadline First) and `schedule_itbs` (Improved Total Bandwidth Server). Also contains `schedule_iot_offloading_online` to simulate time-tick based dispatcher queues. |

| `iot_loader.py` | A utility service. If the user passes a `--json` file to the CLI, this service parses it. It also contains `generate_random_iot_workload` to spawn the periodic and aperiodic tasks for random testing. |

| `task_service.py` | The Database Access Layer (DAL). Wraps SQLAlchemy calls to create, read, and delete `RealTimeTask` records from the SQLite database. |

## 5.4 Models and Data ( `models/` , `data/` )

| File Name                  | Responsibility                                                                                                                                                                                                                         |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>models/iot.py</code> | Defines immutable Python <code>@dataclass</code> objects: <code>IoTDevice</code> , <code>EdgeServer</code> , and <code>IoTTask</code> . Used extensively in memory during simulations for rapid data access without database overhead. |

| File Name                      | Responsibility                                                                                                                                                                                                    |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>models/task.py</code>    | Defines the SQLAlchemy ORM <code>RealTimeTask</code> class mapping to the <code>real_time_tasks</code> SQL table.                                                                                                 |
| <code>data/database.py</code>  | Creates the SQLAlchemy engine and <code>SessionLocal</code> connecting to <code>tasks.db</code> . Contains <code>init_db()</code> to generate tables.                                                             |
| <code>data/seed_data.py</code> | Contains <code>DEFAULT_APERIODIC_TASKS</code> , a list of dictionaries with realistic task names (e.g., "Emergency Brake Audit", "Lidar Map Snapshot") and WCET properties used by the <code>seed</code> command. |

## 5.5 Views ( `views/` )

Views format raw data into human-readable terminal output or image files.

| File Name | Responsibility |

| :--- | :--- |

| `iot_view.py` | Contains `render_iot_schedule` for terminal printouts and `save_iot_gantt_plot` which uses Matplotlib to generate the visually distinct Gantt charts (differentiating Device lanes vs. Server lanes by color). |

| `schedule_view.py` | Generates standard (non-IoT specific) Gantt charts for pure iTBS demonstrations via `save_gantt_plot`. |

| `task_view.py` | Formats database tasks into clean, aligned terminal strings for the `list` command. |

| `clear_view.py` & `seed_view.py` | Minimalist files containing functions that print success/failure messages for database operations (e.g., "Seed complete: created 12 tasks"). |

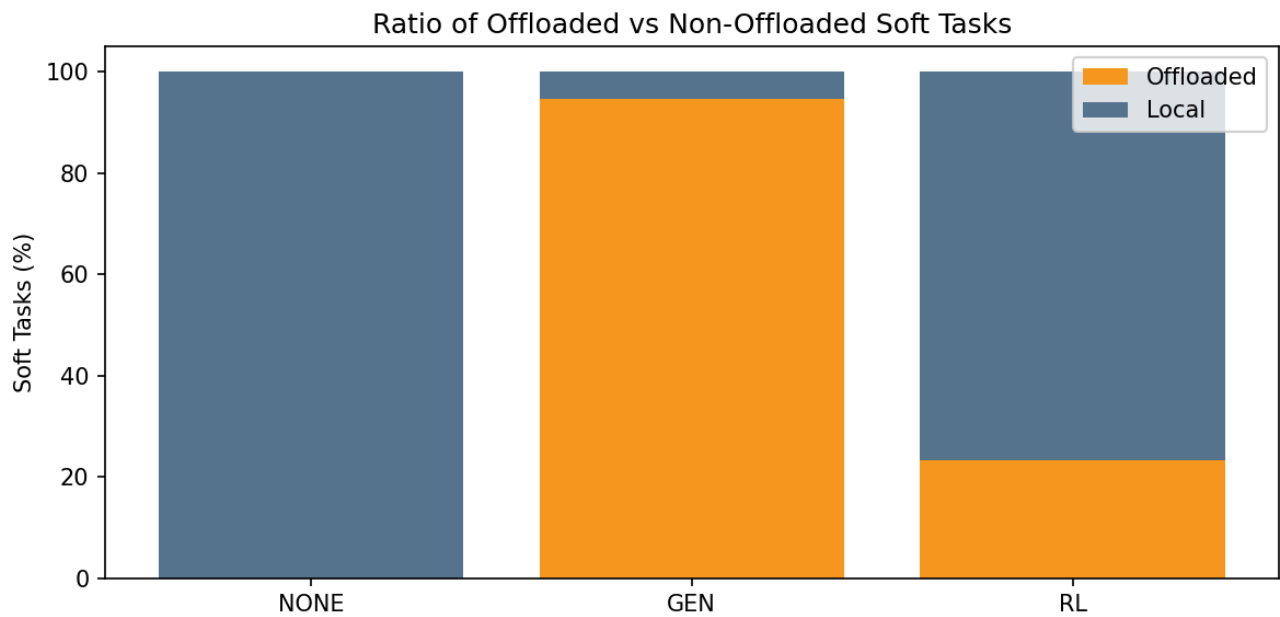
---

## 6. Comparison: Scalability Analysis via `--mode` and `--vars`

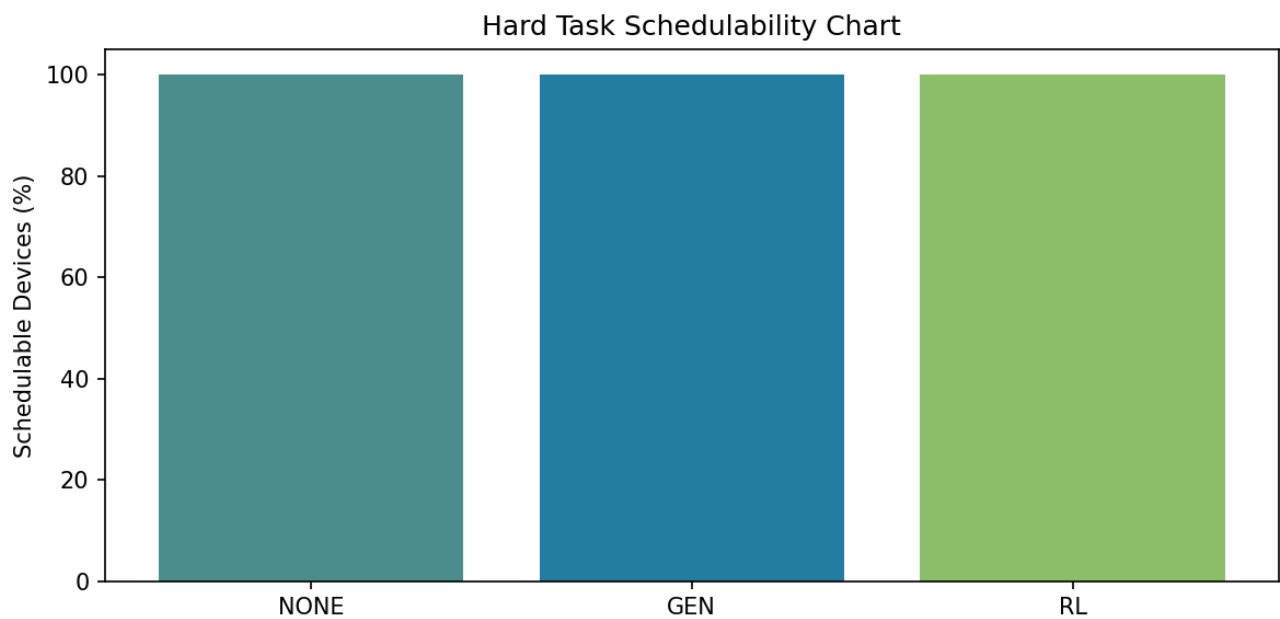
### 6.1

```
python main.py --mode iot --vars 5
```

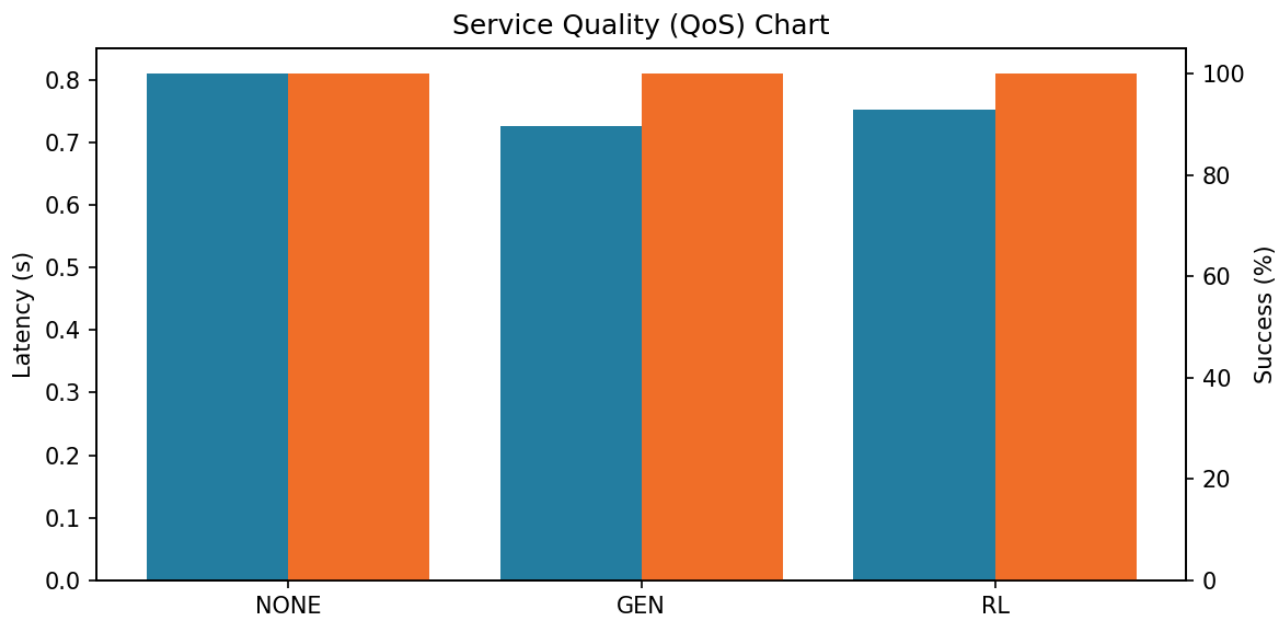
#### 6.1.1: Ratio of offload



### 6.1.2: Offline schedulability of hard tasks

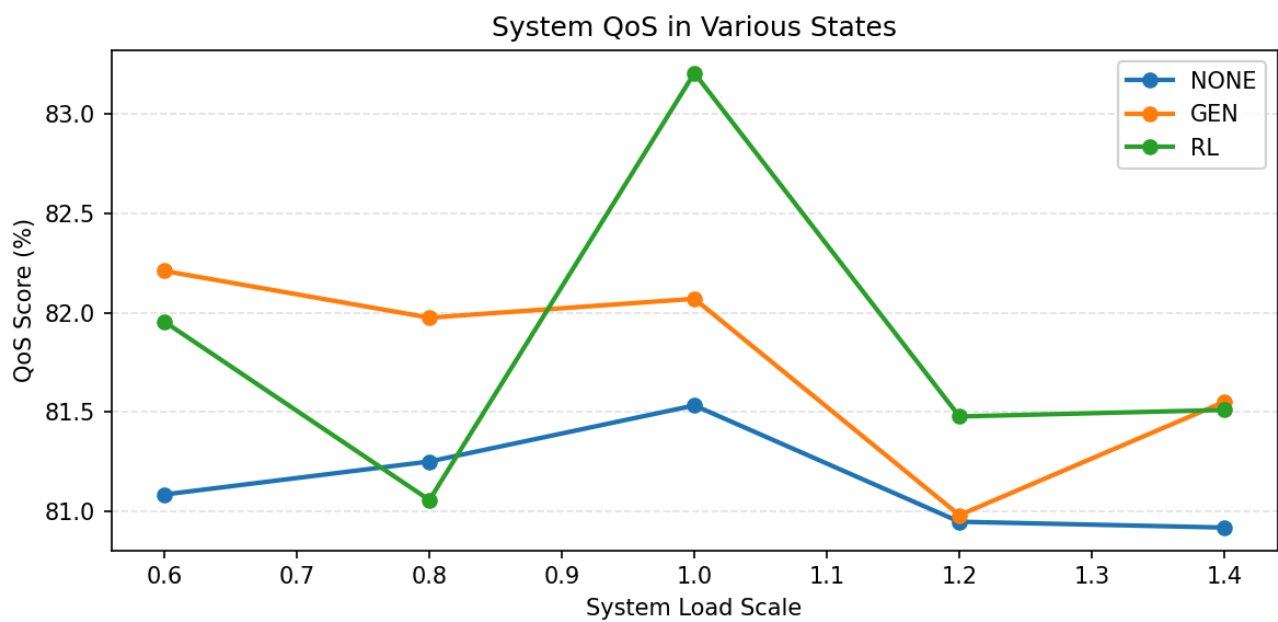


### 6.1.3: Latency when offline/online

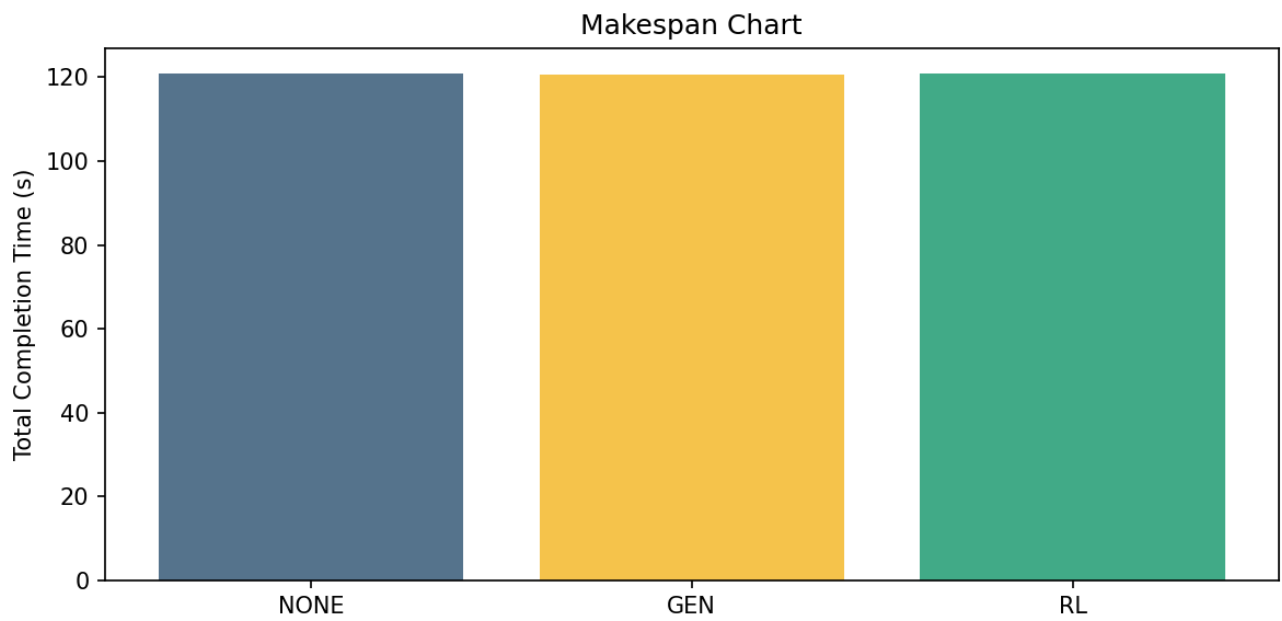


#### 6.1.4: QoS comparison for different algorithms

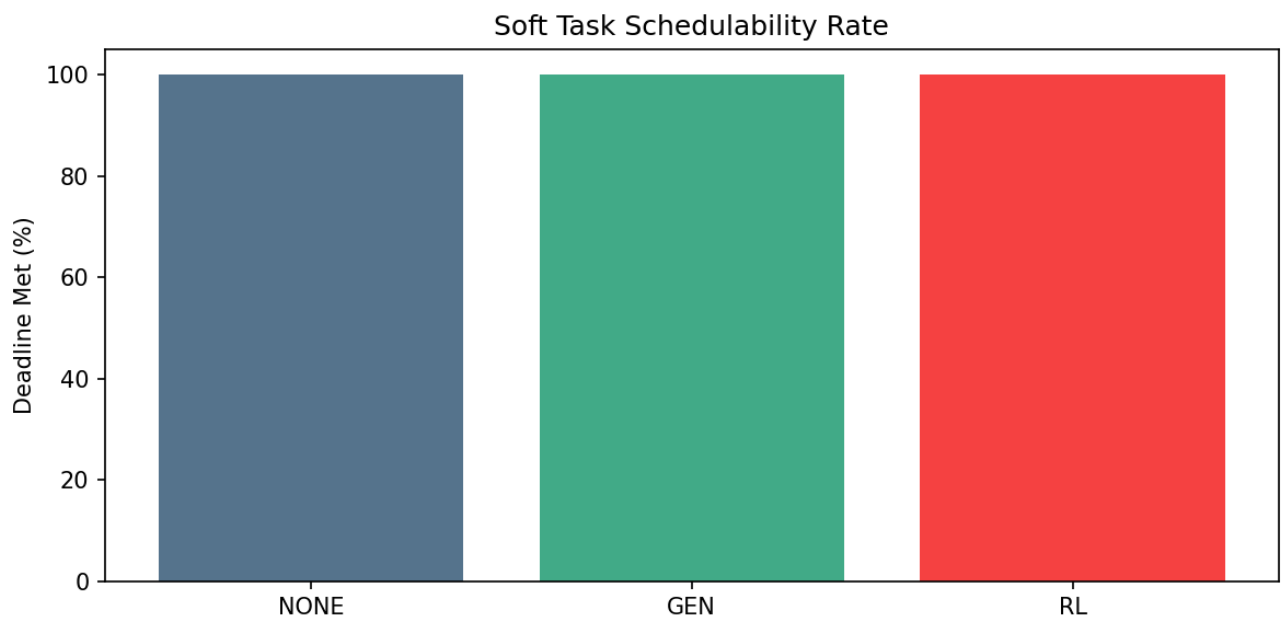
- The RL algorithm performs much better than the other algorithms when the load of the system is 1.
- If the loads increases the performance of RL algorithm decreases but still performs better than the baseline



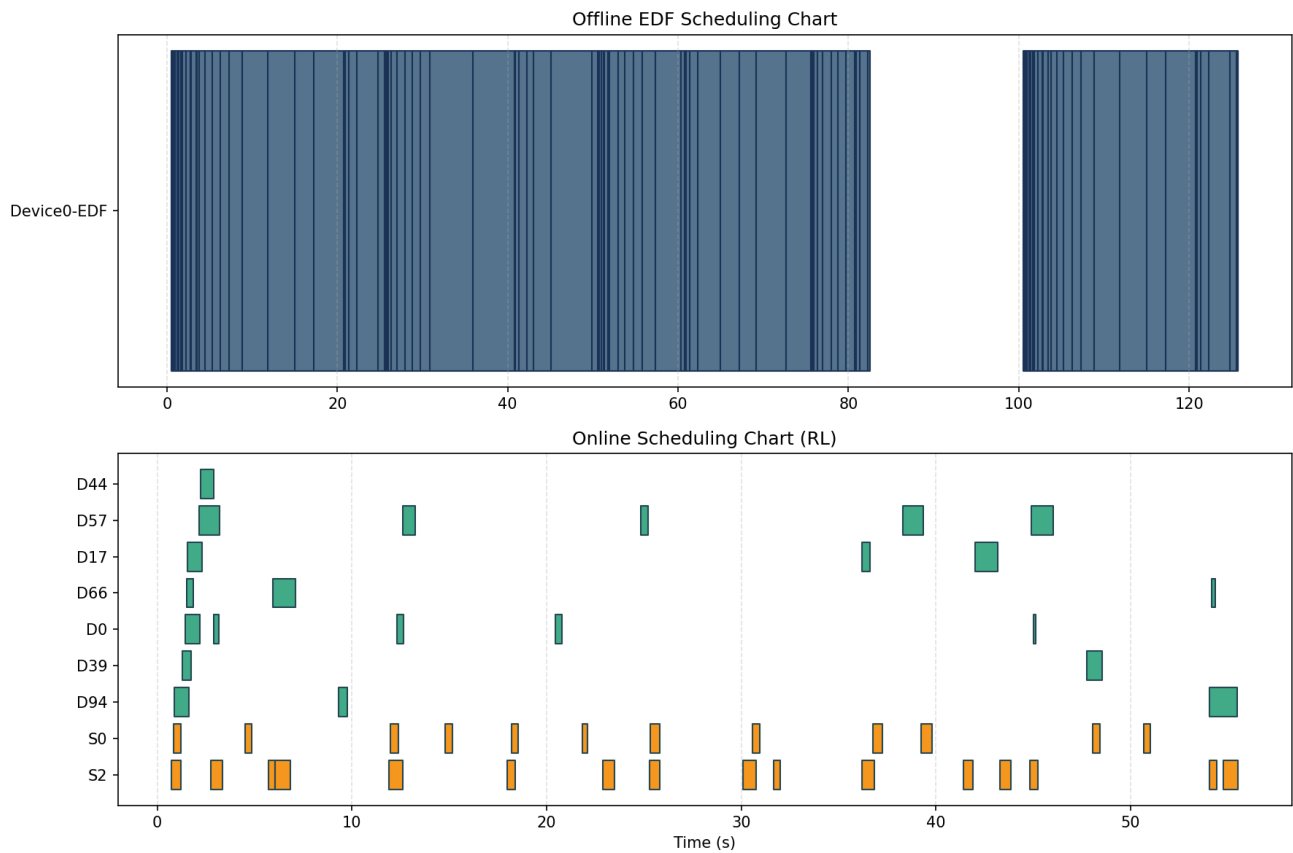
#### 6.1.5: Total time of Run



### 6.1.6: Percentage of deadlines met



### 6.1.7: Schedule of online and offline tasks for the RL algorithm

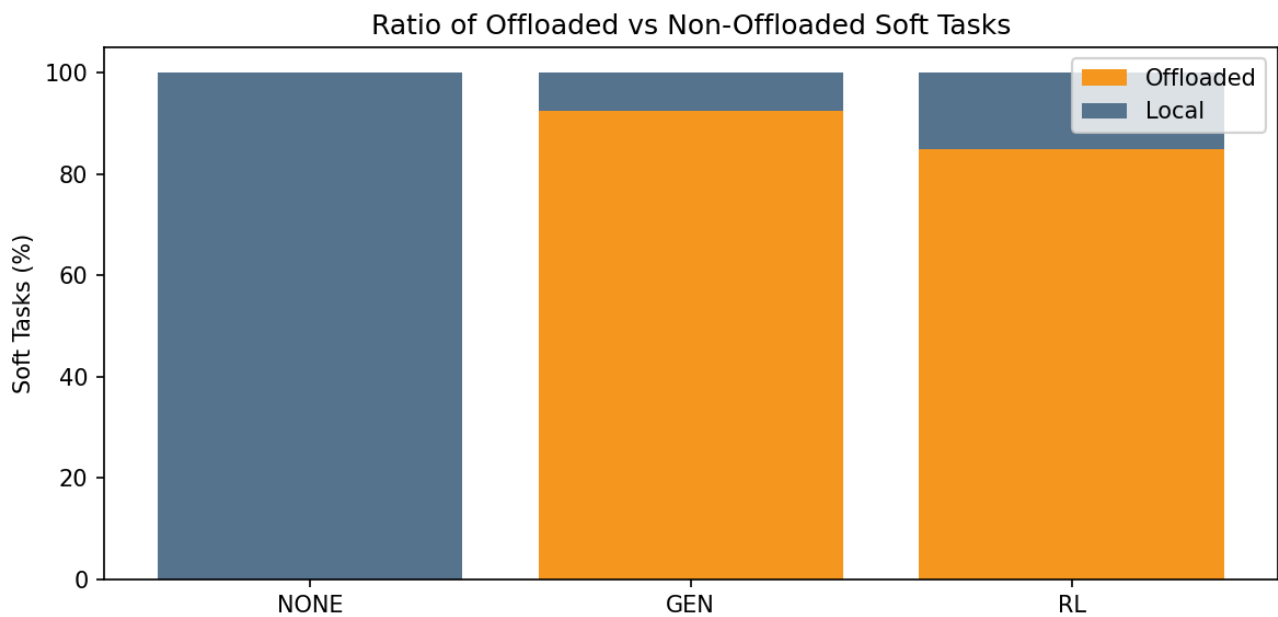


## 6.2

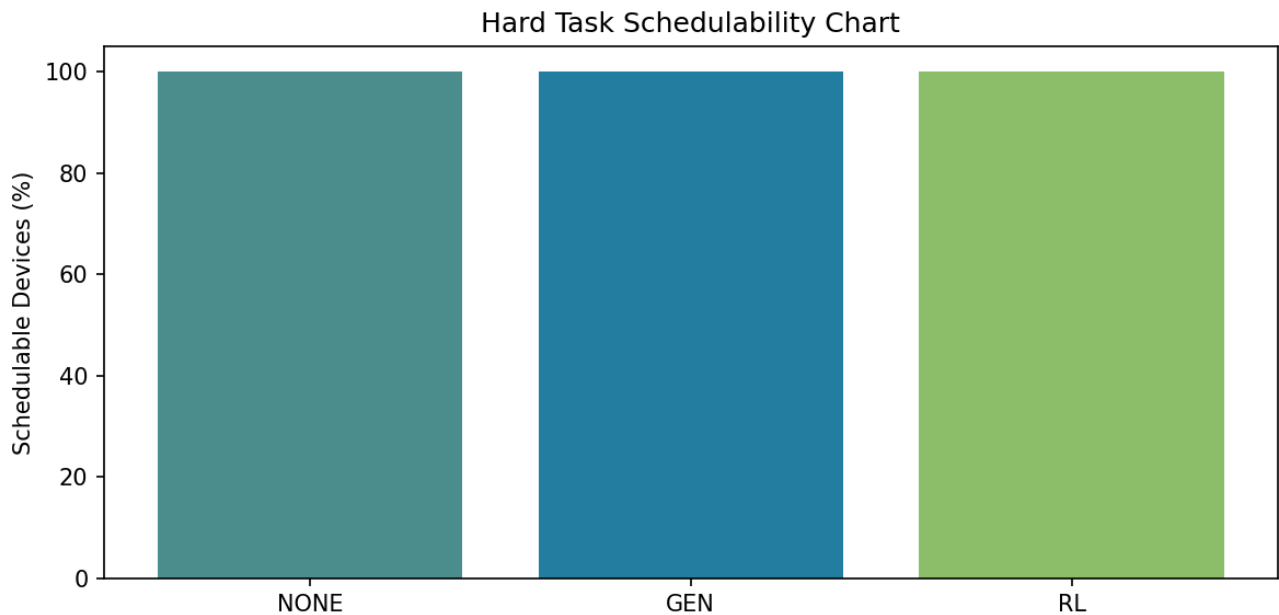
```
python main.py --mode staticserver --vars 10
```

### 6.2.1: Ratio of offload

- offloading is more than the other mode because there are more servers and less IOT devices.

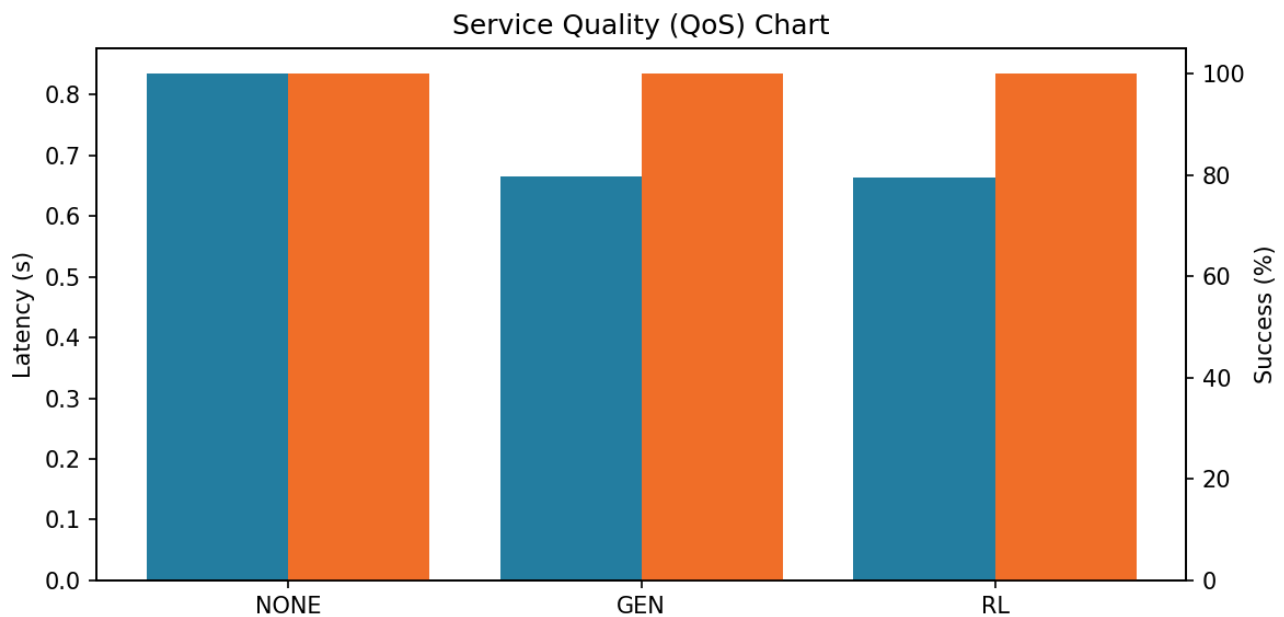


### 6.2.2: Offline schedulability of hard tasks



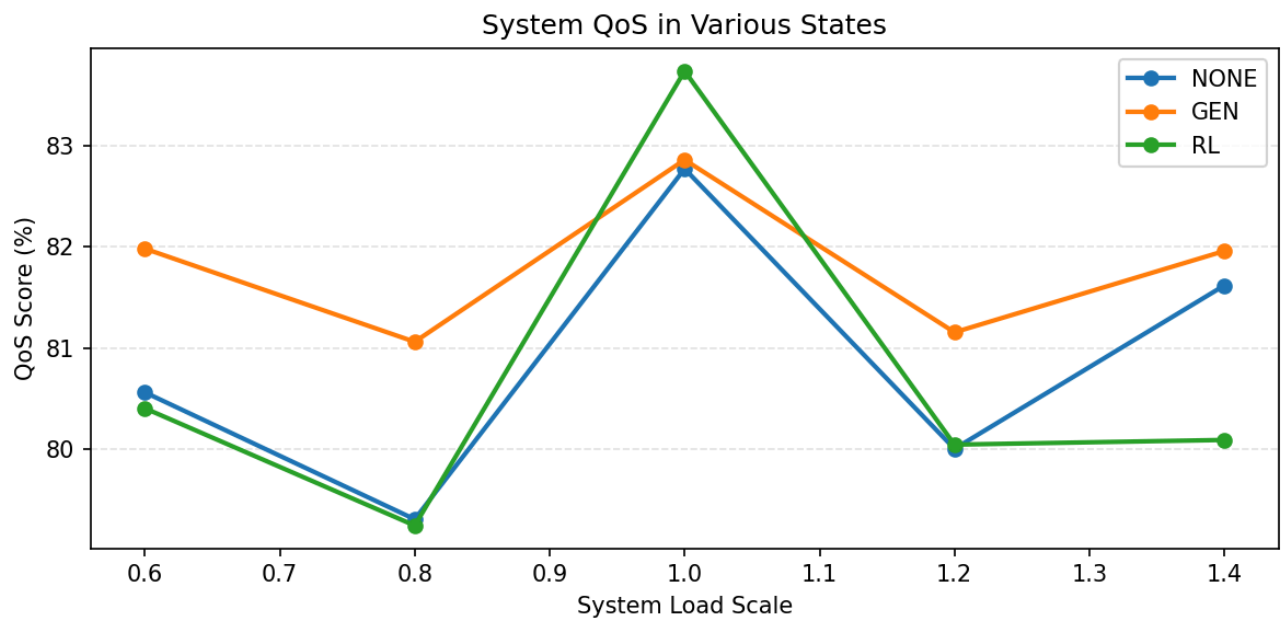
### 6.2.3: Latency when offline/online

- Quality of service declines in Gen and RL because there are more offloading

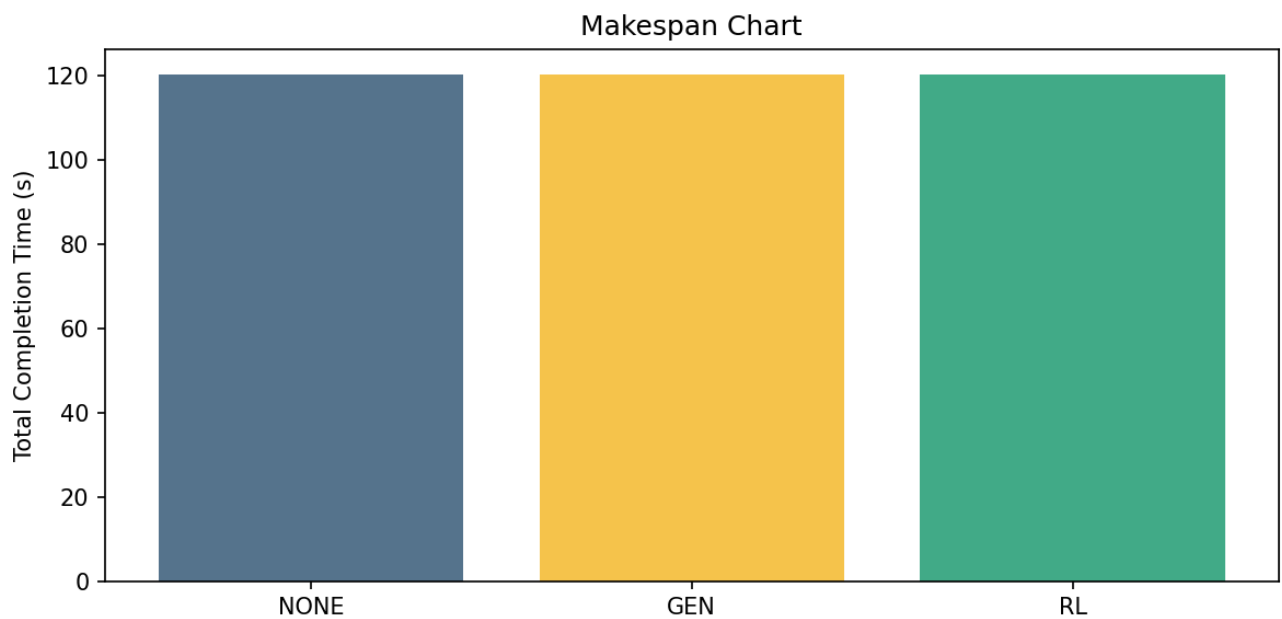


### 6.1.4: QoS comparison for different algorithms

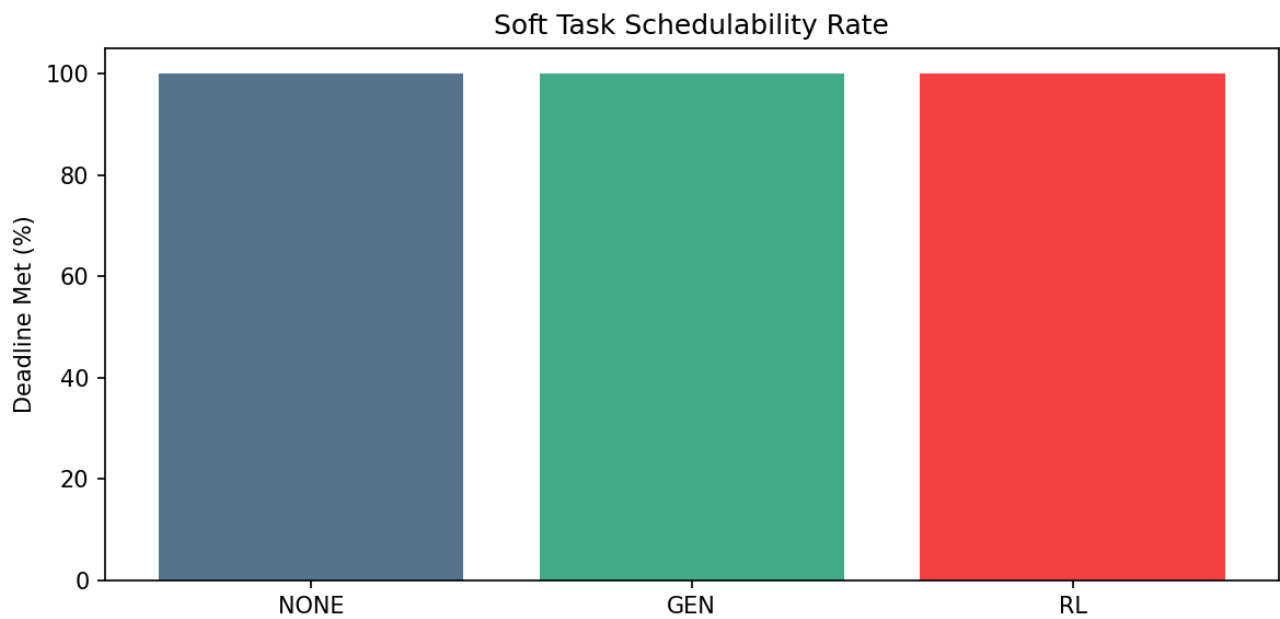
- The RL algorithm performs better than the other algorithms when the load of the system is 1.
- If the loads increases the performance of RL algorithm decreases because the number of offloadings increases more than the avg response time decreases
- Gen algorithm is between the baseline and RL algorithm( In terms of offloading and QoS)
- therefore has a better avg QoS



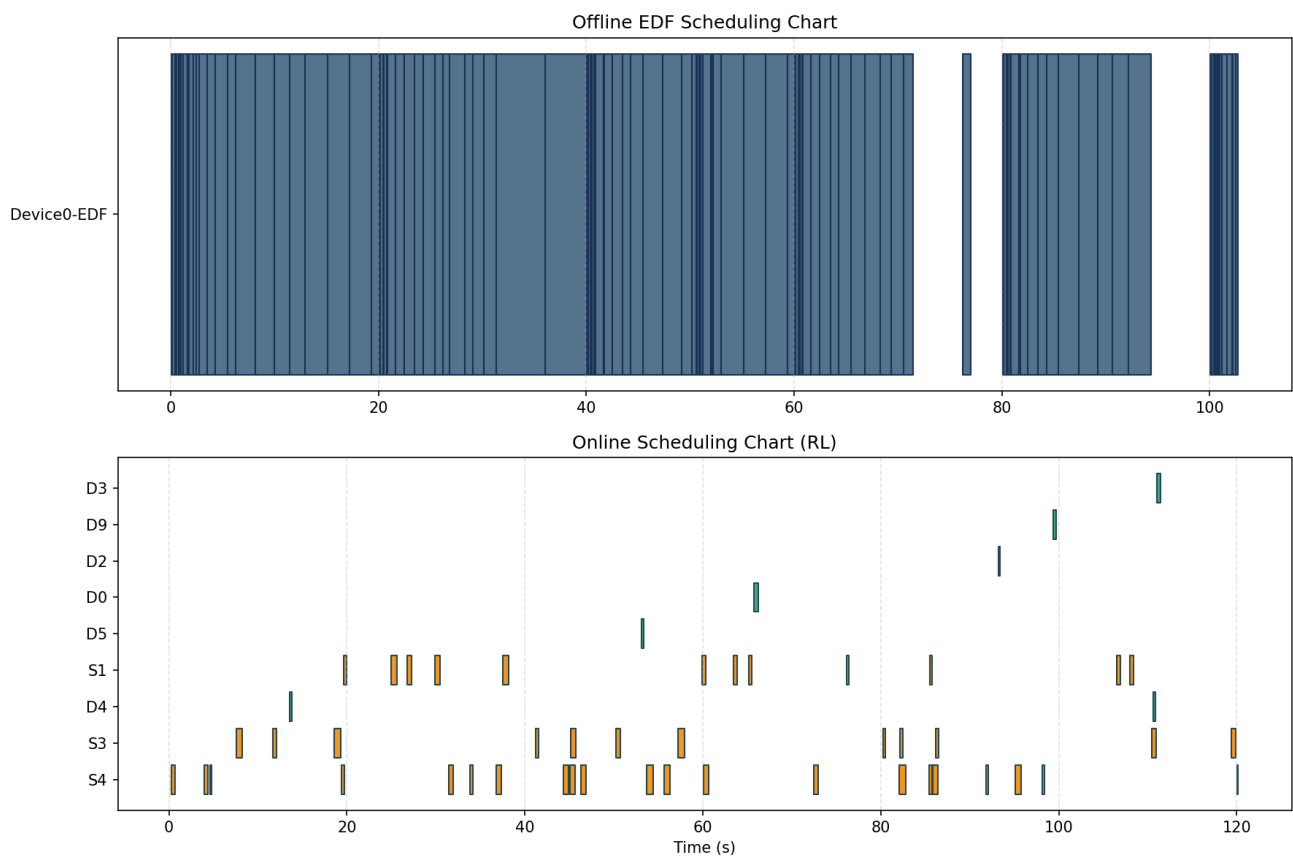
### 6.1.5: Total time of Run



### 6.2.6: Percentage of deadlines met

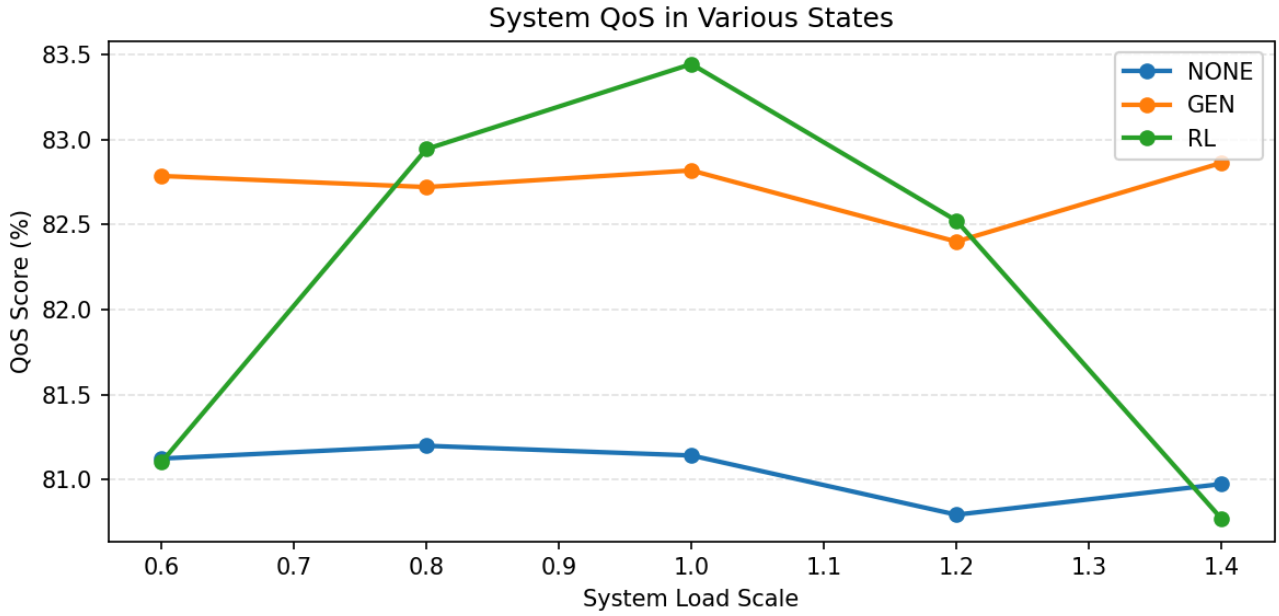


### 6.2.7: Schedule of online and offline tasks for the RL algorithm



## 6.2

```
python main.py --mode iot --vars 10
```



## 6.4

### Horizontal Device Scaling ( `--mode staticserver` )

By fixing the number of Edge Servers to exactly 5 and dynamically increasing the devices (via `--vars`), the framework models **Network Congestion and Server Saturation**. As devices scale from 10 to 50, the demand on the fixed  $U_s$  (Server Bandwidth) increases exponentially. This mode successfully proves that the **RL Agent** is vastly superior to the Baseline; as congestion peaks, the RL agent learns which servers have exhausted their virtual bandwidth and optimally reroutes or holds tasks locally, stabilizing the QoS degradation curve.

### Horizontal Infrastructure Scaling ( `--mode iot` )

Conversely, by fixing the IoT fleet at 100 devices and scaling the Edge Servers (via `--vars`), the framework evaluates the **Diminishing Returns of Hardware**. The output generated by the `run_iot_edge_experiment` clearly illustrates that going from 1 to 5 servers yields a massive spike in the Soft Deadline Success Rate. However, scaling from 10 to 20 servers yields minimal QoS improvement, proving that beyond a certain point, the network transfer overhead (latency) supersedes raw compute availability.